

RÉFÉRENCES ET RESSOURCES POUR LE MÉMOIRE

**Détection Autonome et Distribuée d'Anomalies
(comportements suspects ou signes d'intrusion)
dans les Journaux Systèmes et Réseaux
à l'aide d'Agents Intelligents Multi-Tâches**

1. Titre du Mémoire

**Détection Autonome et Distribuée d'Anomalies
(comportements suspects ou signes d'intrusion) dans
les Journaux Systèmes et Réseaux à l'aide d'Agents
Intelligents Multi-Tâches**

Auteur : Prof. Kyamakya, Bastion-Lab

Destiné à : Étudiant ANDY BITATI

Date : 19 mai 2026

Document de cadrage bibliographique, méthodologique et expérimental

1. Objectif du document

Ce document fournit une base de travail claire et très structurée pour le mémoire d'Andy Bitati. Il identifie 30 références scientifiques utiles et 20 ressources pratiques, puis explique chaque élément dans un style simple, 'for dummies', afin de faciliter la compréhension, le choix méthodologique et la mise en œuvre expérimentale.

Le thème se situe à l'intersection de quatre domaines: la cybersécurité, l'analyse automatique des journaux, l'intelligence artificielle distribuée et les systèmes multi-agents. Le cœur du mémoire est le suivant: collecter des traces systèmes et réseaux, apprendre ce qui est normal, détecter ce qui devient suspect, puis organiser plusieurs agents spécialisés pour travailler ensemble.

2. Lecture rapide du sujet en style simple

Un journal système ou réseau est comparable à un carnet de bord. Chaque ligne raconte une petite action: connexion, erreur, accès refusé, requête réseau, démarrage de service, changement de fichier, etc. Une intrusion laisse souvent des traces, mais ces traces sont noyées dans une grande quantité d'événements normaux. Le défi consiste donc à trouver les signaux faibles sans produire trop de fausses alertes.

Un agent intelligent est un petit module logiciel qui observe, analyse, décide ou communique. Dans ce mémoire, on peut imaginer plusieurs agents: un agent qui collecte les logs, un agent qui les nettoie, un agent qui apprend les comportements normaux, un agent qui détecte les anomalies, un agent qui corrèle les alertes entre machines et un agent qui explique le résultat à l'administrateur.

Le mot 'multi-tâches' signifie que le système ne fait pas une seule chose. Il peut simultanément: détecter des anomalies, classer les alertes, prioriser les événements, expliquer les causes probables, corrélérer plusieurs sources et recommander une action de réponse.

3. Axes scientifiques recommandés

Axe 1 - Fondements IDS et anomalies: Définir intrusion, anomalie, normalité, signature, profil, alerte, faux positif et faux négatif.

Axe 2 - Logs systèmes et réseaux: Comprendre les formats, le parsing, la structuration et la transformation des journaux en événements exploitables.

Axe 3 - Modèles IA/ML: Comparer des baselines simples avec LSTM, autoencodeurs, Transformers, GNN et éventuellement LLMs.

Axe 4 - Agents intelligents distribués: Définir les rôles, l'autonomie, la communication, la coopération, la fusion de scores et la tolérance aux pannes.

Axe 5 - Validation expérimentale: Utiliser datasets publics, métriques correctes, scénarios reproductibles, analyse des limites et discussion des faux positifs.

4. Synthèse rapide des 30 références

N°	Axe	Référence courte	Utilité principale
1	Fondations IDS	Denning, D.	Cette référence sert de socle théorique pour définir intrusion, anomalie, profil normal, alerte et journal d'audit. Elle peut être citée dans l'introduction et dans l'état de l'art...
2	Fondations IDS	Forrest, S.	Le mémoire peut utiliser cette idée pour expliquer la détection d'anomalies au niveau hôte: appels système, journaux d'authentification, erreurs de services, exécutions inhabituel...
3	Fondations IDS	Lee, W.	Cette référence est importante pour la méthodologie. Elle montre que le

			mémoire doit définir une chaîne complète: collecte, préparation, extraction de caractéristiques, apprentissa...
4	Fondations IDS	Axelsson, S.	Cette référence aide à structurer l'état de l'art. Elle permet de placer le mémoire dans la famille des IDS distribués, orientés anomalies, enrichis par des agents intelligents. El...
5	Fondations IDS	Chandola, V.	Cette référence est idéale pour le chapitre 'fondements théoriques'. Elle permet à Andy de définir anomalie ponctuelle, anomalie contextuelle et anomalie collective. Dans les logs,...
6	Fondations IDS	Sommer, R.	Le mémoire doit absolument intégrer ces avertissements. Cette référence sert à discuter les limites, l'évaluation réaliste, la dérive des données, le déséquilibre des classes et l'...
7	ML pour IDS	Buczak, A.	Cette référence est utile pour choisir les baselines du mémoire. Andy peut comparer un modèle simple, comme Random Forest ou SVM, avec une méthode plus avancée, comme LSTM, Transfo...
8	ML pour IDS	Khraisat, A.	Dans le mémoire, cette référence peut servir de pont entre théorie et expérimentation. Elle aide à sélectionner les datasets, à expliquer les limites des datasets historiques, et à...
9	Logs et AIOps	Landauer, M.	Elle est directement alignée avec le sujet du mémoire. Andy peut l'utiliser comme référence principale pour l'état de l'art sur la détection d'anomalies dans les journaux systèmes....
10	Prétraitement des logs	He, P.	Drain peut être un composant essentiel dans le mémoire. Avant de détecter les anomalies, un agent peut utiliser Drain pour transformer les logs bruts en événements structurés. Cela...
11	Détection dans les logs	Du, M.	C'est une référence centrale pour le mémoire. Andy peut reproduire une version simplifiée: parser les logs, créer des séquences, entraîner un modèle à prédire le prochain template,...
12	Détection dans les logs	Meng, W.	Cette référence est utile pour enrichir le mémoire au-delà de la simple prédiction du prochain log. Les agents peuvent surveiller les séquences et les volumes: trop d'échecs de con...
13	Détection dans les logs	Zhang, X.	Cette référence est très utile pour le chapitre limites et robustesse. Dans un vrai système, les agents doivent continuer à fonctionner même quand les formats de logs évoluent. And...
14	Transformers pour logs	Guo, H.	Cette référence permet d'introduire les Transformers dans le mémoire. Elle peut être utilisée comme méthode avancée, après les baselines LSTM. Les agents intelligents multi-tâches ...
15	Sécurité opérationnelle	van Ede, T.	Le mémoire peut s'inspirer de DeepCASE pour proposer des agents qui ne détectent pas seulement, mais qui organisent les alertes. C'est très pertinent pour l'autonomie: un agent

			loc...
16	LLM pour logs	Han, X.	Cette référence permet d'ouvrir le mémoire vers les agents intelligents modernes. Un agent peut utiliser un modèle de langage pour interpréter des logs textuels et générer une expl...
17	LLM pour logs	Guan, W.	Cette référence est utile pour la partie avancée du mémoire: agents multi-tâches capables de lire, classer, expliquer et prioriser des événements. Elle montre une direction récente...
18	NIDS et autoencodeurs	Mirsky, Y.	Le mémoire porte aussi sur les journaux réseaux. Kitsune donne une idée pratique pour un agent réseau: apprendre localement le comportement normal d'un flux et signaler les écarts....
19	Séries temporelles	Zhao, H.	Cette référence est utile si Andy transforme les logs en séries temporelles: nombre d'événements par minute, nombre d'échecs de connexion, nombre de ports contactés, etc. Les agent...
20	Séries temporelles	Su, Y.	Cette référence aide à traiter les métriques extraites des logs: fréquences d'événements, volumes réseau, nombres d'erreurs, latences. Elle peut servir de méthode avancée si le mém...
21	Séries temporelles	Tuli, S.	Cette référence est utile pour une version avancée du prototype. Si les agents collectent plusieurs signaux dans le temps, TranAD donne une piste pour détecter rapidement les anoma...
22	Agents intelligents	Crosbie, M.	Le mémoire peut citer cette référence pour justifier l'architecture distribuée. Elle soutient l'idée que les agents peuvent améliorer la scalabilité, la robustesse et la spécialisa...
23	Agents intelligents	Balasubramaniyan, J.	Cette référence peut devenir une colonne vertébrale architecturale du mémoire. Elle fournit un modèle pour organiser la communication entre agents locaux et agent superviseur. Andy...
24	Agents intelligents	Spafford, E.	Elle est très utile pour discuter pourquoi le mémoire parle d'autonomie et de distribution. Elle aide à écrire les exigences du système: fonctionnement continu, faible supervision ...
25	Agents intelligents	Herrero, A.	Cette référence aide à relier les agents au réseau. Elle est utile pour le mémoire si Andy veut distinguer les agents hôtes et les agents réseaux. Elle donne aussi des éléments pou...
26	Agents intelligents	Bougueroua, N.	Cette référence est directement alignée avec le titre du mémoire. Elle peut servir à définir ce qu'est un IDS collaboratif et à choisir une architecture multi-agents. Elle aide aus...
27	Apprentissage distribué	Khraisat, A.	Cette référence est très utile pour la dimension distribuée et privacy-preserving. Les journaux de sécurité peuvent contenir des données sensibles: IP, utilisateurs, chemins, noms ...

28	Datasets IDS	Moustafa, N.	Andy peut l'utiliser pour tester la partie réseau du mémoire, surtout si le prototype inclut un agent NIDS. Il permet de comparer plusieurs modèles de classification ou d'anomaly d...
29	Datasets IDS	Sharafaldin, I.	Cette référence/dataset est utile pour les expérimentations comparatives. Andy peut l'utiliser pour entraîner et tester des modèles de base. Il faudra aussi mentionner les critique...
30	Datasets IoT/IIoT	Koroniotis, N.	Cette référence ouvre le mémoire vers les systèmes cyberphysiques, IoT et infrastructures distribuées. Elle est intéressante si le Bastion-Lab souhaite relier le mémoire à la sécur...

5. Les 30 références expliquées “for dummies”

Chaque fiche ci-dessous contient une référence, une explication très simple et son utilité concrète pour le mémoire. L'étudiant peut utiliser ces fiches pour rédiger l'état de l'art, construire la méthodologie et choisir les expériences.

Référence 1 - Fondations IDS

Référence complète : Denning, D. E. (1987). An Intrusion-Detection Model. IEEE Transactions on Software Engineering, SE-13(2), 222-232.

Lien principal : <https://www.cs.colostate.edu/~cs656/reading/ieee-se-13-2.pdf>

Explication simple : Ce papier est le point de départ classique des systèmes de détection d'intrusion. L'idée est simple: un système laisse des traces dans des journaux d'audit. Si quelqu'un agit de manière anormale, ses traces doivent différer du comportement habituel. Denning propose donc de surveiller les événements, de construire des profils de comportement normal et de déclencher des alertes quand les événements sortent de ces profils. Pour un étudiant, il faut retenir que la détection d'anomalies ne commence pas avec le deep learning: elle commence avec la question de base suivante: qu'est-ce qui est normal pour cet utilisateur, ce programme ou ce réseau?

Utilité pour le mémoire : Cette référence sert de socle théorique pour définir intrusion, anomalie, profil normal, alerte et journal d'audit. Elle peut être citée dans l'introduction et dans l'état de l'art pour montrer que le mémoire s'inscrit dans une tradition scientifique solide. Elle aide aussi à justifier pourquoi les logs systèmes et réseaux sont une source naturelle de preuve pour détecter des comportements suspects. Dans le mémoire d'Andy, elle peut inspirer la première architecture: collecte de logs, extraction d'événements, construction d'un profil normal, comparaison et alerte.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 2 - Fondations IDS

Référence complète : Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A Sense of Self for Unix Processes. IEEE Symposium on Security and Privacy.

Lien principal : <https://www.cs.unm.edu/~forrest/publications/ieee-sp-96-unix.pdf>

Explication simple : Cette référence introduit une idée très intuitive: un programme informatique a une sorte de comportement habituel, comme une routine. Par exemple, un service Unix normal appelle certaines fonctions système dans un ordre assez stable. Si un attaquant exploite ce programme, l'ordre des appels système peut changer. Les auteurs s'inspirent du système immunitaire: le corps distingue ce qui appartient au soi et ce qui est étranger. Ici, le système apprend le 'soi' d'un programme et signale ce qui paraît étranger.

Utilité pour le mémoire : Le mémoire peut utiliser cette idée pour expliquer la détection d'anomalies au niveau hôte: appels système, journaux d'authentification, erreurs de services, exécutions inhabituelles. Elle est utile pour construire une analogie très pédagogique entre cybersécurité et immunologie artificielle. Pour le thème des agents intelligents, chaque agent peut être responsable de connaître le comportement normal d'un service ou d'un nœud, puis de signaler ce qui ressemble à un 'non-soi'.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 3 - Fondations IDS

Référence complète : Lee, W., Stolfo, S. J., & Mok, K. W. (1999). A Data Mining Framework for Building Intrusion Detection Models. IEEE Symposium on Security and Privacy.

Lien principal : <https://www.cs.columbia.edu/ids/sites/default/files/wenke-ieee99.pdf>

Explication simple : Ce travail explique comment transformer les données de sécurité en un problème de data mining. Au lieu d'écrire uniquement des règles à la main, on extrait automatiquement des motifs fréquents et des caractéristiques utiles à partir des traces système et réseau. Ensuite, on entraîne des modèles capables de reconnaître des intrusions connues ou des

comportements anormaux. En style simple: on prend beaucoup de traces, on cherche les habitudes, puis on apprend à une machine à reconnaître ce qui ne ressemble pas aux habitudes.

Utilité pour le mémoire : Cette référence est importante pour la méthodologie. Elle montre que le mémoire doit définir une chaîne complète: collecte, préparation, extraction de caractéristiques, apprentissage, évaluation. Elle est utile pour expliquer le passage des logs bruts vers des variables exploitables par des agents intelligents. Andy peut s'en servir pour construire un pipeline expérimental clair et pour justifier l'usage de méthodes ML sur les journaux systèmes et réseaux.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 4 - Fondations IDS

Référence complète : Axelsson, S. (2000). Intrusion Detection Systems: A Survey and Taxonomy. Technical Report, Chalmers University of Technology.

Lien principal : <https://www.cse.msu.edu/~cse960/Papers/security/axelsson00intrusion.pdf>

Explication simple : Axelsson propose une classification des IDS. Il distingue notamment les systèmes basés sur les signatures, qui cherchent des attaques déjà connues, et les systèmes basés sur les anomalies, qui cherchent ce qui paraît inhabituel. C'est comme la différence entre reconnaître la photo d'un voleur déjà connu et remarquer qu'une personne se comporte bizarrement dans un bâtiment. Les deux approches sont utiles mais ont des limites: les signatures ratent les attaques nouvelles, les anomalies peuvent générer beaucoup de fausses alertes.

Utilité pour le mémoire : Cette référence aide à structurer l'état de l'art. Elle permet de placer le mémoire dans la famille des IDS distribués, orientés anomalies, enrichis par des agents intelligents. Elle est aussi utile pour discuter les critères importants: source de données, méthode de détection, architecture, réponse, niveau centralisé ou distribué.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 5 - Fondations IDS

Référence complète : Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys, 41(3).

Lien principal : <https://dl.acm.org/doi/10.1145/1541880.1541882>

Explication simple : Ce survey est une carte générale de la détection d'anomalies. Il ne parle pas seulement de cybersécurité: il montre que les anomalies existent dans beaucoup de domaines, par exemple fraude bancaire, panne industrielle, santé ou réseaux. Il explique les grandes familles de méthodes: statistiques, distance, classification, clustering, règles et modèles temporels. En version simple: c'est le manuel qui explique les différentes façons de dire 'ce point ne ressemble pas aux autres'.

Utilité pour le mémoire : Cette référence est idéale pour le chapitre 'fondements théoriques'. Elle permet à Andy de définir anomalie ponctuelle, anomalie contextuelle et anomalie collective. Dans les logs, une seule ligne peut être suspecte, mais une séquence de lignes peut aussi devenir suspecte même si chaque ligne paraît normale isolément. Cette distinction est centrale pour détecter des intrusions discrètes.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 6 - Fondations IDS

Référence complète : Sommer, R., & Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. IEEE Symposium on Security and Privacy.

Lien principal : <https://dl.acm.org/doi/10.1109/SP.2010.25>

Explication simple : Ce papier est très important parce qu'il met en garde contre l'optimisme excessif. En laboratoire, un modèle peut avoir de très bons scores; dans un vrai réseau, les choses sont plus compliquées. Les données changent, les

attaques sont rares, les faux positifs coûtent cher et les attaquants peuvent s'adapter. En style simple: un détecteur qui semble excellent sur un jeu de données propre peut devenir moins utile quand il doit aider un administrateur réel, sous pression, avec beaucoup de bruit.

Utilité pour le mémoire : Le mémoire doit absolument intégrer ces avertissements. Cette référence sert à discuter les limites, l'évaluation réaliste, la dérive des données, le déséquilibre des classes et l'interprétabilité des alertes. Elle aide Andy à éviter un travail purement 'accuracy-driven' et à proposer des métriques plus utiles: rappel, précision, F1, taux de faux positifs, temps de détection et lisibilité des alertes.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 7 - ML pour IDS

Référence complète : Buczak, A. L., & Guven, E. (2016). A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Communications Surveys & Tutorials, 18(2), 1153-1176.

Lien principal : <https://dl.acm.org/doi/10.1109/COMST.2015.2494502>

Explication simple : Ce survey passe en revue les méthodes de machine learning utilisées en intrusion detection: arbres de décision, SVM, réseaux de neurones, clustering, règles, ensembles, etc. Il explique aussi que les données sont aussi importantes que les algorithmes. En style simple: avant de choisir un modèle intelligent, il faut savoir de quelles traces on dispose, ce qu'elles représentent et comment on les transforme en nombres compréhensibles par la machine.

Utilité pour le mémoire : Cette référence est utile pour choisir les baselines du mémoire. Andy peut comparer un modèle simple, comme Random Forest ou SVM, avec une méthode plus avancée, comme LSTM, Transformer ou autoencodeur. Elle aide à justifier une démarche graduelle: commencer par des modèles simples, puis passer aux agents et au deep learning seulement si cela apporte quelque chose.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 8 - ML pour IDS

Référence complète : Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges. Cybersecurity, 2, 20.

Lien principal : <https://link.springer.com/article/10.1186/s42400-019-0038-7>

Explication simple : Ce travail présente un panorama moderne des IDS avec trois angles: techniques, jeux de données et défis. Il est très utile pour comprendre que la détection d'intrusion n'est pas seulement un problème d'algorithme. Il y a aussi la qualité des datasets, les attaques nouvelles, l'équilibre entre détection et fausses alertes, et la capacité à fonctionner en temps réel.

Utilité pour le mémoire : Dans le mémoire, cette référence peut servir de pont entre théorie et expérimentation. Elle aide à sélectionner les datasets, à expliquer les limites des datasets historiques, et à présenter les défis que le mémoire veut adresser: autonomie, distribution, adaptation, réduction des fausses alertes et capacité multi-tâches.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 9 - Logs et AIOps

Référence complète : Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep Learning for Anomaly Detection in Log Data: A Survey. Machine Learning with Applications.

Lien principal : <https://arxiv.org/abs/2207.03820>

Explication simple : Cette référence est centrée sur les logs. Elle explique comment les modèles deep learning sont utilisés pour repérer des anomalies dans des fichiers journaux. Les logs ne sont pas directement des tableaux propres: ce sont des

textes semi-structurés, avec dates, identifiants, messages, erreurs et paramètres. Le survey explique les étapes: parsing, encodage, regroupement en séquences, modèle, score d'anomalie, alerte.

Utilité pour le mémoire : Elle est directement alignée avec le sujet du mémoire. Andy peut l'utiliser comme référence principale pour l'état de l'art sur la détection d'anomalies dans les journaux systèmes. Elle permet aussi de choisir une architecture expérimentale: parser les logs, créer des séquences, entraîner un modèle et interpréter les résultats.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 10 - Prétraitement des logs

Référence complète : He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An Online Log Parsing Approach with Fixed Depth Tree. IEEE ICWS.

Lien principal : https://jiemingzhu.github.io/pub/pihe_icws2017.pdf

Explication simple : Drain résout un problème très concret: les logs sont souvent du texte libre, mais les modèles ont besoin de structures. Drain transforme des lignes comme 'User 123 failed login from IP X' en un modèle de message, par exemple 'User <*> failed login from IP <*>'. Ainsi, on garde l'événement important et on remplace les détails variables par des jokers. C'est comme trier des phrases différentes dans des familles de phrases identiques.

Utilité pour le mémoire : Drain peut être un composant essentiel dans le mémoire. Avant de détecter les anomalies, un agent peut utiliser Drain pour transformer les logs bruts en événements structurés. Cela rend les séquences plus courtes, plus lisibles et plus faciles à apprendre. C'est aussi utile pour expliquer les alertes à un humain: on peut montrer quel template est devenu anormal.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 11 - Détection dans les logs

Référence complète : Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. ACM CCS.

Lien principal : <https://dl.acm.org/doi/10.1145/3133956.3134015>

Explication simple : DeepLog traite les logs comme une langue. Si une séquence normale de logs ressemble à une phrase correcte, alors une intrusion peut ressembler à une phrase bizarre. Le modèle LSTM apprend à prédire le prochain événement de log. Si l'événement réel n'est pas parmi les événements probables, le système soupçonne une anomalie. En style simple: DeepLog apprend 'ce qui vient normalement après quoi'.

Utilité pour le mémoire : C'est une référence centrale pour le mémoire. Andy peut reproduire une version simplifiée: parser les logs, créer des séquences, entraîner un modèle à prédire le prochain template, puis déclencher une alerte quand la séquence devient improbable. Dans une architecture multi-agents, chaque agent peut apprendre les séquences normales d'un service ou d'une machine.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 12 - Détection dans les logs

Référence complète : Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., & Zhou, R. (2019). LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. IJCAI.

Lien principal : <https://www.ijcai.org/proceedings/2019/658>

Explication simple : LogAnomaly améliore l'idée de DeepLog en regardant deux dimensions: l'ordre des événements et leur quantité. Parfois, le problème n'est pas seulement qu'un événement arrive au mauvais moment; c'est aussi qu'un événement arrive trop souvent ou pas assez. LogAnomaly utilise des représentations sémantiques des templates pour mieux comprendre les relations entre événements.

Utilité pour le mémoire : Cette référence est utile pour enrichir le mémoire au-delà de la simple prédiction du prochain log. Les agents peuvent surveiller les séquences et les volumes: trop d'échecs de connexion, trop de connexions SSH, trop d'erreurs d'autorisation, etc. Cela aide à détecter des comportements suspects progressifs qui ne se voient pas dans une seule ligne de log.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 13 - Détection dans les logs

Référence complète : Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.-G., & Chintalapati, M. (2019). Robust Log-Based Anomaly Detection on Unstable Log Data. ACM ESEC/FSE.

Lien principal : <https://dl.acm.org/doi/10.1145/3338906.3338931>

Explication simple : LogRobust part d'une observation réaliste: les logs changent souvent. Les développeurs modifient les messages, ajoutent des informations, changent les services. Un modèle trop rigide risque de considérer toute nouveauté comme une attaque. LogRobust cherche donc à être plus robuste en représentant les événements avec du sens, pas seulement avec des identifiants fixes.

Utilité pour le mémoire : Cette référence est très utile pour le chapitre limites et robustesse. Dans un vrai système, les agents doivent continuer à fonctionner même quand les formats de logs évoluent. Andy peut utiliser cette idée pour proposer un agent capable d'apprendre progressivement et de distinguer une vraie intrusion d'un simple changement logiciel.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 14 - Transformers pour logs

Référence complète : Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log Anomaly Detection via BERT. IJCNN.

Lien principal : <https://par.nsf.gov/servlets/purl/10317798>

Explication simple : LogBERT applique l'idée de BERT aux logs. BERT apprend le contexte en masquant certaines parties d'une séquence et en essayant de les retrouver. Pour les logs, cela signifie apprendre quelles entrées vont naturellement ensemble. Si le modèle n'arrive pas à reconstruire correctement une séquence ou si une séquence paraît très improbable, elle peut être anormale.

Utilité pour le mémoire : Cette référence permet d'introduire les Transformers dans le mémoire. Elle peut être utilisée comme méthode avancée, après les baselines LSTM. Les agents intelligents multi-tâches pourraient utiliser des embeddings de type BERT pour mieux comprendre le contexte d'un événement et réduire les faux positifs.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 15 - Sécurité opérationnelle

Référence complète : van Ede, T., Aghakhani, H., Spahn, N., Bortolameotti, R., Cova, M., Continella, A., van Steen, M., Peter, A., Kruegel, C., & Vigna, G. (2022). DEEPCASE: Semi-Supervised Contextual Analysis of Security Events. IEEE Symposium on Security and Privacy.

Lien principal : <https://doi.org/10.1109/SP46214.2022.9833671>

Explication simple : DeepCASE cherche à aider les analystes sécurité qui reçoivent trop d'alertes. Il apprend le contexte autour des événements et regroupe les séquences similaires. Ensuite, un humain peut décider quelles séquences sont importantes, inutiles ou dangereuses. En style simple: au lieu de montrer mille alertes isolées, le système regroupe les histoires qui se ressemblent.

Utilité pour le mémoire : Le mémoire peut s'inspirer de DeepCASE pour proposer des agents qui ne détectent pas seulement, mais qui organisent les alertes. C'est très pertinent pour l'autonomie: un agent local peut filtrer et regrouper, tandis qu'un agent superviseur peut présenter les cas importants à l'administrateur.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 16 - LLM pour logs

Référence complète : Han, X., Yuan, S., & Trabelsi, M. (2023). LogGPT: Log Anomaly Detection via GPT. IEEE International Conference on Big Data.

Lien principal : <https://arxiv.org/abs/2309.14482>

Explication simple : LogGPT applique les modèles de type GPT aux logs. L'idée ressemble à la prédiction du mot suivant: le modèle apprend à prédire le prochain événement de log. Mais les auteurs adaptent aussi l'apprentissage pour que le modèle devienne meilleur dans la détection d'anomalies. En simple: un GPT spécialisé apprend la grammaire des logs normaux et repère les phrases techniques bizarres.

Utilité pour le mémoire : Cette référence permet d'ouvrir le mémoire vers les agents intelligents modernes. Un agent peut utiliser un modèle de langage pour interpréter des logs textuels et générer une explication lisible. Andy doit toutefois rester prudent: un LLM peut coûter cher, halluciner ou être difficile à valider. Cette référence est donc utile pour une extension ou une discussion prospective.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 17 - LLM pour logs

Référence complète : Guan, W., Cao, J., Qian, S., Gao, J., & Ouyang, C. (2024). LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv.

Lien principal : <https://arxiv.org/abs/2411.08561>

Explication simple : LogLLM combine des idées de BERT et de Llama pour mieux comprendre le sens des messages de logs. Il cherche à éviter une dépendance trop forte aux parsers classiques, car les logs sont du texte semi-naturel. En style simple: au lieu de réduire chaque log à un simple code, le système essaie de comprendre le message lui-même.

Utilité pour le mémoire : Cette référence est utile pour la partie avancée du mémoire: agents multi-tâches capables de lire, classer, expliquer et prioriser des événements. Elle montre une direction récente: utiliser les LLMs non seulement comme chatbots, mais comme outils de compréhension sémantique des journaux techniques.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 18 - NIDS et autoencodeurs

Référence complète : Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. NDSS.

Lien principal : https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-3_Mirsky_paper.pdf

Explication simple : Kitsune est un système de détection d'intrusion réseau qui utilise plusieurs petits autoencodeurs. Un autoencodeur apprend à reconstruire les données normales. Quand il reconstruit mal une observation, cela signifie que cette observation ne ressemble pas au normal. L'intérêt de Kitsune est qu'il peut apprendre en ligne, de manière relativement légère, même sur des ressources limitées.

Utilité pour le mémoire : Le mémoire porte aussi sur les journaux réseaux. Kitsune donne une idée pratique pour un agent réseau: apprendre localement le comportement normal d'un flux et signaler les écarts. Cette référence est utile pour comparer une approche log-séquence avec une approche autoencodeur sur trafic réseau.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 19 - Séries temporelles

Référence complète : Zhao, H., Wang, Y., Duan, J., Huang, C., Cao, D., Tong, Y., Xu, B., Bai, J., Tong, J., & Zhang, Q. (2020). Multivariate Time-Series Anomaly Detection via Graph Attention Network. IEEE ICDM.

Lien principal : <https://arxiv.org/abs/2009.02040>

Explication simple : MTAD-GAT traite les données comme des séries temporelles multivariées: plusieurs mesures évoluent ensemble. Le modèle utilise des graphes d'attention pour apprendre quelles variables sont liées. Par exemple, dans un système, le nombre d'erreurs, la charge CPU, les connexions réseau et les tentatives d'authentification peuvent être corrélés.

Utilité pour le mémoire : Cette référence est utile si Andy transforme les logs en séries temporelles: nombre d'événements par minute, nombre d'échecs de connexion, nombre de ports contactés, etc. Les agents peuvent alors surveiller les relations entre plusieurs signaux, au lieu d'analyser chaque variable séparément.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 20 - Séries temporelles

Référence complète : Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., & Pei, D. (2019). Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. KDD.

Lien principal : <https://dl.acm.org/doi/10.1145/3292500.3330672>

Explication simple : OmniAnomaly modélise les séries temporelles complexes avec un réseau récurrent probabiliste. L'idée est de capturer le comportement normal même quand les données sont bruitées et incertaines. En simple: le modèle apprend les variations normales d'un système et déclenche une alerte quand les mesures deviennent trop improbables.

Utilité pour le mémoire : Cette référence aide à traiter les métriques extraites des logs: fréquences d'événements, volumes réseau, nombres d'erreurs, latences. Elle peut servir de méthode avancée si le mémoire ajoute des signaux temporels en plus des messages de logs.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 21 - Séries temporelles

Référence complète : Tuli, S., Casale, G., & Jennings, N. R. (2022). TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. PVLDB.

Lien principal : <https://arxiv.org/abs/2201.07284>

Explication simple : TranAD utilise des Transformers pour détecter des anomalies dans des séries temporelles. Les Transformers sont capables de regarder plusieurs positions d'une séquence et de comprendre quelles parties sont importantes. En style simple: le modèle ne regarde pas seulement le dernier événement; il regarde le contexte plus large pour comprendre pourquoi quelque chose est suspect.

Utilité pour le mémoire : Cette référence est utile pour une version avancée du prototype. Si les agents collectent plusieurs signaux dans le temps, TranAD donne une piste pour détecter rapidement les anomalies et diagnostiquer quelles variables ont contribué au problème.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 22 - Agents intelligents

Référence complète : Crosbie, M., & Spafford, E. H. (1995). Defending a Computer System Using Autonomous Agents. Purdue Technical Report.

Lien principal : <https://docs.lib.purdue.edu/cgi/viewcontent.cgi?article=2199&context=cstech>

Explication simple : Cette référence est très proche du thème des agents. Les auteurs proposent de ne pas construire un IDS comme un gros bloc central, mais comme un ensemble de petits agents autonomes. Chaque agent observe une partie du système et participe à la défense globale. En style simple: au lieu d'avoir un seul gardien pour tout le campus, on place plusieurs petits gardiens spécialisés dans différents bâtiments.

Utilité pour le mémoire : Le mémoire peut citer cette référence pour justifier l'architecture distribuée. Elle soutient l'idée que les agents peuvent améliorer la scalabilité, la robustesse et la spécialisation. Andy peut s'en inspirer pour définir des agents par rôle: agent collecteur, agent parser, agent détecteur, agent corrélateur, agent explicateur et agent de réponse.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 23 - Agents intelligents

Référence complète : Balasubramanian, J. S., Garcia-Fernandez, J. O., Isacoff, D., Spafford, E., & Zamboni, D. (1998). An Architecture for Intrusion Detection Using Autonomous Agents. ACSAC.

Lien principal : <https://zzamboni.org/files/pubs/aafid-acsac98.pdf>

Explication simple : Ce papier présente l'architecture AAFID, une architecture d'intrusion detection fondée sur des agents autonomes. Les agents font de la collecte et de l'analyse à bas niveau, puis remontent l'information vers des niveaux supérieurs. En simple: chaque agent surveille un petit morceau du système, et les résultats sont coordonnés pour comprendre la situation globale.

Utilité pour le mémoire : Cette référence peut devenir une colonne vertébrale architecturale du mémoire. Elle fournit un modèle pour organiser la communication entre agents locaux et agent superviseur. Andy peut adapter cette architecture aux logs modernes: logs Linux, Windows, réseau, Wazuh, Zeek, Suricata, etc.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 24 - Agents intelligents

Référence complète : Spafford, E. H., & Zamboni, D. (2000). Intrusion Detection Using Autonomous Agents. Computer Networks, 34(4), 547-570.

Lien principal : <https://people.scs.carleton.ca/~soma/id-2006w/readings/spafford-agents.pdf>

Explication simple : Ce travail approfondit l'idée d'AAFID et discute les bénéfices des agents: distribution, tolérance aux pannes, configurabilité et spécialisation. Les agents peuvent être simples et nombreux, ce qui permet de ne pas dépendre d'un seul composant central. En version simple: si un agent tombe, tout le système ne s'effondre pas; les autres continuent de travailler.

Utilité pour le mémoire : Elle est très utile pour discuter pourquoi le mémoire parle d'autonomie et de distribution. Elle aide à écrire les exigences du système: fonctionnement continu, faible supervision humaine, adaptation, communication entre agents et résistance aux pannes.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 25 - Agents intelligents

Référence complète : Herrero, A., & Corchado, E. (2009). Multiagent Systems for Network Intrusion Detection: A Review. In Computational Intelligence in Security for Information Systems.

Lien principal : https://gicap.ubu.es/publications/2009/PDF/2009_c05_Multiagent_Systems_for_Network.pdf

Explication simple : Cette revue se concentre sur les systèmes multi-agents pour la détection d'intrusion réseau. Elle montre que les agents peuvent être utilisés pour surveiller, communiquer, coopérer et parfois visualiser les menaces. En style simple: un agent seul peut voir une partie du problème; plusieurs agents qui partagent leurs observations peuvent voir une attaque plus globale.

Utilité pour le mémoire : Cette référence aide à relier les agents au réseau. Elle est utile pour le mémoire si Andy veut distinguer les agents hôtes et les agents réseaux. Elle donne aussi des éléments pour discuter la coopération: comment fusionner les alertes, comment éviter les doublons, comment décider qu'un comportement local devient une menace globale.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 26 - Agents intelligents

Référence complète : Bougueroua, N., Mazouzi, S., Belaoued, M., Seddari, N., Derhab, A., & Bouras, A. (2021). A Survey on Multi-Agent Based Collaborative Intrusion Detection Systems. Journal of Artificial Intelligence and Soft Computing Research.

Lien principal : <https://sciendo.com/pdf/10.2478/jaiscr-2021-0008>

Explication simple : Ce survey récent classe les systèmes collaboratifs de détection d'intrusion basés sur les agents. Il explique comment les agents peuvent coopérer, échanger des informations et participer à une décision collective. En simple: ce n'est pas seulement plusieurs détecteurs séparés; c'est un système où les détecteurs se parlent pour réduire les erreurs.

Utilité pour le mémoire : Cette référence est directement alignée avec le titre du mémoire. Elle peut servir à définir ce qu'est un IDS collaboratif et à choisir une architecture multi-agents. Elle aide aussi à discuter les décisions distribuées: vote, fusion de scores, corrélation d'alertes, confiance entre agents et hiérarchie.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 27 - Apprentissage distribué

Référence complète : Khraisat, A., et al. (2024). Survey on Federated Learning for Intrusion Detection System. ACM Computing Surveys.

Lien principal : <https://dl.acm.org/doi/full/10.1145/3687124>

Explication simple : L'apprentissage fédéré permet d'entraîner des modèles sur plusieurs sites sans envoyer les données brutes au serveur central. Chaque site apprend localement, puis partage seulement des paramètres ou des mises à jour de modèle. En simple: les données restent chez elles, mais les agents apprennent ensemble.

Utilité pour le mémoire : Cette référence est très utile pour la dimension distribuée et privacy-preserving. Les journaux de sécurité peuvent contenir des données sensibles: IP, utilisateurs, chemins, noms de machines. L'apprentissage fédéré peut être proposé comme extension pour permettre à plusieurs machines ou organisations de coopérer sans partager tous les logs bruts.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 28 - Datasets IDS

Référence complète : Moustafa, N., & Slay, J. (2015). UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. MilCIS.

Lien principal : <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Explication simple : UNSW-NB15 est un dataset de trafic réseau qui contient du trafic normal et plusieurs types d'attaques modernes. Il a été créé pour remplacer des datasets plus anciens, moins réalistes. En style simple: c'est une collection d'exemples où l'on sait déjà quels flux sont normaux et quels flux correspondent à une attaque.

Utilité pour le mémoire : Andy peut l'utiliser pour tester la partie réseau du mémoire, surtout si le prototype inclut un agent NIDS. Il permet de comparer plusieurs modèles de classification ou d'anomaly detection. Il faut cependant discuter ses limites: aucun dataset public ne remplace totalement un réseau réel.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 29 - Datasets IDS

Référence complète : Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISP. Dataset CICIDS2017.

Lien principal : <https://www.unb.ca/cic/datasets/ids-2017.html>

Explication simple : CICIDS2017 est un dataset très utilisé pour évaluer les systèmes de détection d'intrusion. Il contient du trafic normal et des scénarios d'attaque comme DoS, DDoS, brute force, botnet et attaques web. En simple: c'est un terrain d'entraînement public pour apprendre à reconnaître des comportements réseau malveillants.

Utilité pour le mémoire : Cette référence/dataset est utile pour les expérimentations comparatives. Andy peut l'utiliser pour entraîner et tester des modèles de base. Il faudra aussi mentionner les critiques méthodologiques connues: certains datasets contiennent des biais ou des erreurs de labellisation. Cela renforce l'importance d'une évaluation prudente.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

Référence 30 - Datasets IoT/IloT

Référence complète : Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. Future Generation Computer Systems; and TON_IoT dataset resources from UNSW Canberra.

Lien principal : <https://research.unsw.edu.au/projects/bot-iot-dataset>

Explication simple : Bot-IoT et TON_IoT sont utiles pour étudier des attaques dans des environnements IoT/IloT. Ces environnements ont beaucoup d'objets connectés, de capteurs, de services et de communications. En style simple: ce sont des datasets pour comprendre comment des attaques se manifestent dans des réseaux d'objets connectés, pas seulement dans des réseaux informatiques classiques.

Utilité pour le mémoire : Cette référence ouvre le mémoire vers les systèmes cyberphysiques, IoT et infrastructures distribuées. Elle est intéressante si le Bastion-Lab souhaite relier le mémoire à la sécurité des systèmes intelligents, smart grids, robots, transport ou bâtiments connectés. Les agents peuvent être placés sur différents nœuds IoT ou passerelles pour détecter localement des comportements suspects.

Comment l'exploiter concrètement : Lire l'introduction et la méthode, relever les définitions importantes, puis résumer en 5 à 8 lignes dans l'état de l'art. Si la référence propose un algorithme ou un dataset, l'utiliser comme baseline, comparaison ou justification méthodologique.

6. Ressources utiles: datasets, codes, outils et bases de connaissances

Les ressources suivantes permettent de passer d'un mémoire uniquement théorique à un mémoire expérimental. Elles sont classées de manière pratique: données, codes de parsing/analyse, plateformes de sécurité et bases de connaissances.

Ressource 1 - LogHub (Dataset logs systèmes)

Lien : <https://github.com/logpai/loghub>

Utilité simple : Collection publique de logs pour l'analyse automatique. Utile pour commencer vite avec HDFS, BGL, Thunderbird et autres logs sans devoir collecter soi-même des données. Pour le mémoire, c'est la source la plus directe pour tester le parsing, les séquences de logs et la détection d'anomalies.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 2 - LogHub-2.0 (Dataset annoté / benchmark parsing)

Lien : <https://github.com/logpai/loghub-2.0>

Utilité simple : Version plus structurée et plus grande pour évaluer les parsers de logs. Utile si Andy veut comparer Drain avec d'autres méthodes de parsing. Permet de montrer que le prétraitement est une étape scientifique, pas un simple nettoyage.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 3 - AIT AECID anomaly-detection-log-datasets (Scripts + datasets logs)

Lien : <https://github.com/ait-aecid/anomaly-detection-log-datasets>

Utilité simple : Réunit des scripts pour préparer plusieurs datasets de logs utilisés en recherche. Très pratique pour éviter de perdre du temps dans la mise en forme. Utile pour obtenir des séquences prêtes à être utilisées par des modèles LSTM, Transformer ou agents.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 4 - ADFA IDS Datasets (Dataset appels système HIDS)

Lien : <https://research.unsw.edu.au/projects/adfa-ids-datasets>

Utilité simple : Jeux de données Linux et Windows basés sur des appels système. Très utiles pour la détection au niveau hôte. Un agent peut apprendre les séquences normales d'appels système et repérer les comportements suspects.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 5 - AWSCTD (Dataset appels système Windows)

Lien : <https://github.com/DiPasco/AWSCTD>

Utilité simple : Dataset orienté traces d'appels système Windows causées par des activités malveillantes. Utile si le mémoire veut couvrir des environnements Windows en plus de Linux. À utiliser surtout pour une extension expérimentale.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 6 - UNSW-NB15 (Dataset intrusion réseau)

Lien : <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Utilité simple : Dataset réseau avec plusieurs familles d'attaques. Très bon candidat pour entraîner un agent réseau et comparer des baselines classiques. Utile pour apprendre la différence entre trafic normal et trafic malveillant.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 7 - CICIDS2017 (Dataset intrusion réseau)

Lien : <https://www.unb.ca/cic/datasets/ids-2017.html>

Utilité simple : Très utilisé dans les travaux IDS. Utile pour reproduire des résultats et comparer les modèles. Attention: il faut discuter les biais possibles et éviter de conclure trop vite à partir d'un seul dataset.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 8 - CSE-CIC-IDS2018 (Dataset cloud/réseau/logs)

Lien : <https://www.unb.ca/cic/datasets/ids-2018.html>

Utilité simple : Dataset plus grand avec trafic réseau et logs dans une infrastructure réaliste. Utile pour tester la scalabilité et la capacité distribuée. Il peut soutenir un scénario multi-agents sur plusieurs machines.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 9 - CTU-13 (Dataset botnet)

Lien : <https://www.stratosphereips.org/datasets-ctu13>

Utilité simple : Dataset de trafic botnet réel mélangé à du trafic normal. Très utile pour travailler sur comportements persistants, communications C2 et anomalies réseau. Bon complément aux datasets CIC et UNSW.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 10 - Bot-IoT (Dataset IoT botnet)

Lien : <https://research.unsw.edu.au/projects/bot-iot-dataset>

Utilité simple : Dataset orienté IoT avec trafic normal et botnet. Utile si le mémoire veut démontrer que les agents peuvent surveiller des environnements d'objets connectés ou de systèmes cyberphysiques.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 11 - TON_IoT (Dataset IoT/IloT multi-source)

Lien : <https://research.unsw.edu.au/projects/toniot-datasets>

Utilité simple : Dataset IoT/IloT comprenant réseau, télémétrie et logs systèmes. Très intéressant pour un thème multi-tâches, car les agents peuvent analyser plusieurs types de données en même temps.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 12 - LogPai Logparser (Code parsing logs)

Lien : <https://github.com/logpai/logparser>

Utilité simple : Boîte à outils de parsing automatique. Utile pour tester Drain et d'autres parsers sans les recoder. C'est une excellente base de prototype pour passer des logs bruts aux événements structurés.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 13 - Drain3 (Code parsing streaming)

Lien : <https://github.com/logpai/Drain3>

Utilité simple : Implémentation pratique de Drain pour extraire des templates en ligne. Utile pour un agent collecteur/parser qui travaille en continu sur un flux de logs.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 14 - Loglizer (Code ML pour anomalies logs)

Lien : <https://github.com/logpai/loglizer>

Utilité simple : Toolkit de machine learning pour la détection d'anomalies dans les logs. Utile pour construire rapidement des baselines classiques et comparer avec une approche agents/deep learning.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 15 - Deep-loglizer (Code deep learning logs)

Lien : <https://github.com/logpai/deep-loglizer>

Utilité simple : Extension deep learning de Loglizer. Utile pour expérimenter des modèles plus modernes sans repartir de zéro. Peut servir de base pour une version avancée du prototype.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 16 - DeepCASE (Code semi-supervisé sécurité)

Lien : <https://github.com/Thijsvanede/DeepCASE>

Utilité simple : Implémentation de DeepCASE pour analyser les séquences d'événements de sécurité. Utile pour l'idée de regroupement d'alertes et de réduction du travail de l'analyste.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 17 - Wazuh (SIEM/XDR open source)

Lien : <https://wazuh.com/>

Utilité simple : Plateforme open source pour collecter, corréliser et analyser des événements de sécurité. Très utile pour créer un environnement expérimental réaliste avec agents installés sur des machines.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 18 - Zeek (Network Security Monitoring)

Lien : <https://zeek.org/>

Utilité simple : Outil open source de surveillance réseau qui produit des logs riches sur DNS, HTTP, connexions, fichiers, etc. Utile pour alimenter l'agent réseau avec des journaux exploitables.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 19 - Suricata (IDS/IPS réseau open source)

Lien : <https://suricata.io/>

Utilité simple : Moteur IDS/IPS et d'analyse réseau. Utile pour générer des alertes et logs réseau qui pourront être comparés aux alertes produites par les agents intelligents.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

Ressource 20 - MITRE ATT&CK + Sigma Rules (Base de connaissances + règles)

Lien : <https://attack.mitre.org/> ; <https://github.com/SigmaHQ/sigma>

Utilité simple : ATT&CK aide à relier une alerte à une technique d'attaque connue. Sigma fournit des règles de détection génériques pour les logs. Ensemble, ils aident à rendre les alertes plus compréhensibles et plus proches des pratiques SOC.

Usage conseillé : Commencer par une installation locale ou un petit échantillon. Documenter la version, la taille des données, les champs utilisés et les limites. Ne jamais mélanger données d'entraînement et données de test.

7. Méthodologie recommandée pour le mémoire

Étape 1 - Définir le périmètre: Choisir un périmètre réaliste: logs Linux, logs Windows, logs réseau Zeek/Suricata, ou un dataset public. Éviter de vouloir tout faire dès le départ.

Étape 2 - Construire le pipeline de données: Collecter les logs, supprimer les champs inutiles, parser les messages, transformer les logs en templates et créer des séquences temporelles.

Étape 3 - Mettre en place des baselines: Tester au moins un modèle simple: Isolation Forest, Random Forest, SVM, Logistic Regression, LOF ou autoencodeur. Cela donne un point de comparaison.

Étape 4 - Ajouter l'intelligence séquentielle: Tester une méthode de séquence comme LSTM/DeepLog, puis éventuellement Transformer/LogBERT/LogGPT selon la capacité technique disponible.

Étape 5 - Concevoir les agents: Définir les agents par rôle: collecteur, parser, détecteur local, corrélateur, superviseur, explicateur. Décrire leurs entrées, sorties et messages échangés.

Étape 6 - Évaluer correctement: Utiliser précision, rappel, F1, matrice de confusion, taux de faux positifs, temps de détection et coût de calcul. Ne pas se limiter à l'accuracy.

Étape 7 - Expliquer les alertes: Associer chaque alerte à une séquence de logs, un score, une source, une hypothèse de cause et, si possible, une tactique MITRE ATT&CK.

8. Architecture conceptuelle proposée

Une architecture cohérente pour ce mémoire peut être organisée en six couches. La couche 1 collecte les logs. La couche 2 parse et nettoie les messages. La couche 3 extrait des caractéristiques: templates, fréquences, séquences, fenêtres temporelles, adresses IP, ports, utilisateurs. La couche 4 contient les agents détecteurs locaux. La couche 5 réalise la corrélation distribuée. La couche 6 produit l'explication, la priorisation et la réponse.

Dans cette architecture, un agent local ne doit pas forcément tout comprendre. Son rôle est de surveiller une zone précise. L'intelligence globale vient de la coopération: les agents partagent des scores, des résumés et des alertes afin de détecter une attaque distribuée ou progressive.

Agent	Entrées	Sorties	Tâche principale
Agent collecteur	Logs bruts	Flux normalisé	Récupérer les événements systèmes et réseaux.
Agent parser	Lignes de logs	Templates + paramètres	Transformer le texte libre en structure exploitable.
Agent détecteur local	Séquences et features	Score d'anomalie	Apprendre le normal et signaler les écarts.
Agent corrélateur	Alertes locales	Incident probable	Fusionner plusieurs signaux faibles.
Agent explicateur	Incident + contexte	Explication lisible	Dire pourquoi l'alerte est importante.
Agent superviseur	Toutes les sorties	Décision/priorité	Orchestrer les agents et gérer les priorités.

9. Métriques et pièges à éviter

Accuracy: À utiliser avec prudence: si 99% des données sont normales, un modèle peut sembler excellent en prédisant toujours normal.

Précision: Parmi les alertes déclenchées, combien sont vraiment des anomalies? Très important pour réduire la fatigue des analystes.

Rappel: Parmi les attaques réelles, combien sont détectées? Très important pour ne pas rater des intrusions.

F1-score: Compromis entre précision et rappel. Utile quand les classes sont déséquilibrées.

Faux positifs par heure/jour: Métrique opérationnelle très concrète: combien d'alertes inutiles un humain doit-il vérifier?

Temps de détection: Combien de temps entre le début du comportement suspect et l'alerte?

Explicabilité: L'alerte doit indiquer les logs, les variables ou les agents qui ont contribué à la décision.

10. Proposition de plan pour l'état de l'art

- Introduction à la cybersécurité, aux IDS et aux logs.
- Détection d'anomalies: concepts, types d'anomalies, métriques et limites.
- Analyse des journaux systèmes: parsing, templates, séquences, modèles statistiques et deep learning.
- Détection d'intrusion réseau: datasets, NIDS, autoencodeurs et flux réseau.
- Systèmes multi-agents: autonomie, coopération, architecture distribuée et fusion d'alertes.
- Approches récentes: Transformers, LLMs, apprentissage fédéré, explicabilité et SOC augmentés.
- Synthèse critique: ce que le mémoire garde, ce qu'il simplifie, ce qu'il évalue expérimentalement.

11. Glossaire minimal

Log/Journaux: Trace écrite d'un événement produit par un système, une application ou un réseau.

Anomalie: Comportement qui ne ressemble pas au comportement attendu ou normal.

Intrusion: Action non autorisée ou malveillante visant un système.

IDS: Système de détection d'intrusion.

HIDS: IDS basé sur l'hôte: fichiers, processus, appels système, logs locaux.

NIDS: IDS basé sur le réseau: paquets, flux, ports, protocoles, connexions.

Parsing: Transformation d'un log textuel en structure exploitable.

Template de log: Forme générale d'un message de log après remplacement des paramètres variables.

Agent intelligent: Module logiciel capable d'observer, analyser, décider, communiquer ou apprendre.

Faux positif: Alerte générée alors qu'il n'y a pas de vraie attaque.

Faux négatif: Attaque réelle non détectée.

12. Conclusion opérationnelle pour Andy Bitati

Le mémoire doit rester ambitieux mais maîtrisé. La bonne stratégie est de construire un prototype progressif: d'abord un pipeline de logs simple, ensuite un détecteur local, puis une architecture multi-agents. Les méthodes avancées comme Transformers, LLMs ou apprentissage fédéré peuvent être présentées comme extensions si le temps ne permet pas une implémentation complète.

La contribution la plus importante ne sera pas seulement d'obtenir un bon score, mais de montrer une architecture claire, compréhensible, reproductible et utile pour un environnement réel. Le mémoire doit donc combiner théorie, expérimentation, critique des datasets, explication des alertes et discussion des limites.

13. Bibliographie compacte et liens

- [1] Denning, D. E. (1987). An Intrusion-Detection Model. IEEE Transactions on Software Engineering, SE-13(2), 222-232. [Lien](#)
- [2] Forrest, S., Hofmeyr, S. A., Somayaji, A., & Longstaff, T. A. (1996). A Sense of Self for Unix Processes. IEEE Symposium on Security and Privacy. [Lien](#)
- [3] Lee, W., Stolfo, S. J., & Mok, K. W. (1999). A Data Mining Framework for Building Intrusion Detection Models. IEEE Symposium on Security and Privacy. [Lien](#)
- [4] Axelsson, S. (2000). Intrusion Detection Systems: A Survey and Taxonomy. Technical Report, Chalmers University of Technology. [Lien](#)
- [5] Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys, 41(3). [Lien](#)
- [6] Sommer, R., & Paxson, V. (2010). Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. IEEE Symposium on Security and Privacy. [Lien](#)
- [7] Buczak, A. L., & Guven, E. (2016). A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Communications Surveys & Tutorials, 18(2), 1153-1176. [Lien](#)
- [8] Khraisat, A., Gondal, I., Vamplew, P., & Kamruzzaman, J. (2019). Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges. Cybersecurity, 2, 20. [Lien](#)

- [9] Landauer, M., Onder, S., Skopik, F., & Wurzenberger, M. (2023). Deep Learning for Anomaly Detection in Log Data: A Survey. Machine Learning with Applications. [Lien](#)
- [10] He, P., Zhu, J., Zheng, Z., & Lyu, M. R. (2017). Drain: An Online Log Parsing Approach with Fixed Depth Tree. IEEE ICWS. [Lien](#)
- [11] Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. ACM CCS. [Lien](#)
- [12] Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., & Zhou, R. (2019). LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. IJCAI. [Lien](#)
- [13] Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.-G., & Chintalapati, M. (2019). Robust Log-Based Anomaly Detection on Unstable Log Data. ACM ESEC/FSE. [Lien](#)
- [14] Guo, H., Yuan, S., & Wu, X. (2021). LogBERT: Log Anomaly Detection via BERT. IJCNN. [Lien](#)
- [15] van Ede, T., Aghakhani, H., Spahn, N., Bortolameotti, R., Cova, M., Continella, A., van Steen, M., Peter, A., Kruegel, C., & Vigna, G. (2022). DEEPCASE: Semi-Supervised Contextual Analysis of Security Events. IEEE Symposium on Security and Privacy. [Lien](#)
- [16] Han, X., Yuan, S., & Trabelsi, M. (2023). LogGPT: Log Anomaly Detection via GPT. IEEE International Conference on Big Data. [Lien](#)
- [17] Guan, W., Cao, J., Qian, S., Gao, J., & Ouyang, C. (2024). LogLLM: Log-based Anomaly Detection Using Large Language Models. arXiv. [Lien](#)
- [18] Mirsky, Y., Doitshman, T., Elovici, Y., & Shabtai, A. (2018). Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. NDSS. [Lien](#)
- [19] Zhao, H., Wang, Y., Duan, J., Huang, C., Cao, D., Tong, Y., Xu, B., Bai, J., Tong, J., & Zhang, Q. (2020). Multivariate Time-Series Anomaly Detection via Graph Attention Network. IEEE ICDM. [Lien](#)
- [20] Su, Y., Zhao, Y., Niu, C., Liu, R., Sun, W., & Pei, D. (2019). Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. KDD. [Lien](#)
- [21] Tuli, S., Casale, G., & Jennings, N. R. (2022). TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. PVLDB. [Lien](#)
- [22] Crosbie, M., & Spafford, E. H. (1995). Defending a Computer System Using Autonomous Agents. Purdue Technical Report. [Lien](#)
- [23] Balasubramaniyan, J. S., Garcia-Fernandez, J. O., Isacoff, D., Spafford, E., & Zamboni, D. (1998). An Architecture for Intrusion Detection Using Autonomous Agents. ACSAC. [Lien](#)
- [24] Spafford, E. H., & Zamboni, D. (2000). Intrusion Detection Using Autonomous Agents. Computer Networks, 34(4), 547-570. [Lien](#)
- [25] Herrero, A., & Corchado, E. (2009). Multiagent Systems for Network Intrusion Detection: A Review. In Computational Intelligence in Security for Information Systems. [Lien](#)
- [26] Bougueroua, N., Mazouzi, S., Belaoued, M., Seddari, N., Derhab, A., & Bouras, A. (2021). A Survey on Multi-Agent Based Collaborative Intrusion Detection Systems. Journal of Artificial Intelligence and Soft Computing Research. [Lien](#)
- [27] Khraisat, A., et al. (2024). Survey on Federated Learning for Intrusion Detection System. ACM Computing Surveys. [Lien](#)
- [28] Moustafa, N., & Slay, J. (2015). UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems. MilCIS. [Lien](#)
- [29] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. ICISSP. Dataset CICIDS2017. [Lien](#)
- [30] Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset. Future Generation Computer Systems; and TON_IoT dataset resources from UNSW Canberra. [Lien](#)